

2026 “钉耙编程”中国大学生算法设计春季联赛（10）题解

By 杭州电子科技大学出题组

2026 年 5 月 29 日

1001 歪歪吃水果

考虑最小数量的水果的放置。将 $n \times n$ 的区域划分成 $m \times m$ 个小块，对于每个小块我们放置在右下角是最优的，故如果有多余部分未被划分到，则也会被右小角小块给覆盖影响。故最小数量水果的下限为 $\lfloor \frac{n}{m} \rfloor^2$ 。

如果小于这个下限则为 NO，否则为 YES。

1002 歪歪魔法树

可以发现，满足条件 $LCA_{i,j} = LCA_{i,k} = LCA_{j,k}$ 的三元组 (i, j, k) 有两种情况：

情况一：同一个节点的三棵子树中的三个不同节点。

情况二：三个不同节点中有一个节点是其余两点的 LCA ，并且其余两点是这个节点的两棵子树中的不同节点。

故我们可以枚举可作为该三元组的 LCA 节点，计算贡献。

1002 歪歪魔法树

即问题转化成：

$$\sum_u \sum_{i < j < k, (i, j, k) \in \text{subTree}(u)} S_i S_j S_k + \sum_u \sum_{i < j, (i, j) \in \text{subTree}(u)} a_u S_i S_j$$

其中

$$S_u = \sum_{i \in \text{subTree}(u)} a_i$$

1002 歪歪魔法树

故对第一项每棵子树 $subTree(u)$, 有恒等式:

$$\sum_{i < j < k, (i, j, k) \in subTree(u)} S_i S_j S_k = \frac{s_1^3 - 3s_1 s_2 + 2s_3}{6}$$

其中

$$s_1 = \sum_{i \in subTree(u)} S_i$$

$$s_2 = \sum_{i \in subTree(u)} S_i^2$$

$$s_3 = \sum_{i \in subTree(u)} S_i^3$$

1002 歪歪魔法树

而第二项固定 a_u ，使用常规遍历处理技巧即可算出。
故一遍遍历算出总和即可，时间复杂度 $O(n)$ 。

1003 月亮棋

由于棋盘大小固定为 3×3 ，我们可以直接通过暴力枚举来解决。

- 若 $n < 2$ ：场上棋子太少，显然落子后无法凑齐 3 个子胜出，直接判定无解。
- 若 $n = 2$ 或 $n = 3$ ：从小到大枚举每一个格子作为落子点：
- **合法点判定**：落子点必须是没有被任何棋子占据的空位。注意，当 $n = 3$ 时，你最久远的那个旧棋子在落子那一刻仍然占据着格子，因此该位置**不能**作为新子的落点。
- **胜利判定**：假设在此处落子，遍历其所在的行、列以及可能存在的对角线。若某一条线上其余 2 个位置皆为己方保留的棋子，则可以立即胜利。

由于双重循环按行列从 1 到 3 遍历，第一个满足条件的合法点即为行列坐标最小的解。

1004 歪歪爱追剧

很明显，题目是由两部分构成的。

第一步显然是将符合条件的区间筛选出来。由于 L 过大，所以我们考虑二分。对于每个区间 $[l_i, r_i]$ 我们可以通过两个端点在 a 数组中的次序大小关系，来看他们之间是否至少包含一个 a_x ，从而筛选出符合条件的区间。

1004 歪歪爱追剧

第二步，对符合条件的区间采用动态规划解决问题。设 dp_i 表示电视剧进行到第 i 分钟能观看的最大分钟数。故将区间按照右端点进行排序之后，很容易写出转移方程：

$$dp[line[i].r] = \max_{0 \leq j \leq line[i].l-1} ((dp[j]) + (line[i].r - line[i].l + 1))$$

由于右端点是不断递增的，故我们使用前缀 max 优化即可优化掉对 j 那一维的枚举，做到 $O(n)$ 的时间复杂度。然而考虑到 L 非常大，但实际上我们只用到了区间左右端点这两个数值，故对其进行离散化之后就可照上述进行求解。总时间复杂度 $O(n \log m)$ 。

1005 搭桥

考虑枚举右端点 $i \in [1, n]$ 寻找最优 h ，维护总代价最小值，总代价函数 $F(h)$ 由两端点调整费、中间木桩高于 h 部分的削减费及飞行费 $(n - i)k$ 组成。

显然最优高度 h 必然存在于原高度集合中，通过离散化 a_i ，将值域搜索转化为 $O(n)$ 规模的下标搜索。同时由于各项代价均为关于 h 的线性或分段线性函数， $F(h)$ 具有凸性，可以对有序数组进行三分搜索或针对斜率（导数）二分查找。

对于斜率二分，令 s 为当前斜率的正负，若 $s \geq 0$ 说明高度偏高，反之则偏低。为了快速计算高于 h 的木桩数量以及高出部分之和，我们利用树状数组动态维护已遍历木桩的高度，每轮枚举将 a_i 存入树状数组。单次决策复杂度 $O(\log^2 n)$ ，总时间复杂度为 $O(n \log^2 n)$ 。

1006 真的是原根?

令 $n = p - 1$

因为 $x \equiv g^e \pmod{p}$

所以每个 x 和指数 e 一一对应。

因此问题可以转化为统计满足 $\gcd(e, n) = k$ 的指数 e 。

如果 k 不是 n 的因数，那么显然不存在这样的 e ，集合为空，答案为：0 1。

1006 真的是原根?

下面只考虑 $k \mid n$ 的情况。

令 $d = \frac{n}{k}$ 。

若 $\gcd(e, n) = k$

则 e 可以写成: $e = k \cdot t$

代入可得: $\gcd(kt, kd) = k$

也就是: $\gcd(t, d) = 1$

1006 真的是原根?

所以满足条件的指数个数，等于模 d 下与 d 互质的数的个数，即 $\varphi(d)$

因此集合 S_k 的大小为 $|S_k| = \varphi(d)$

1006 真的是原根?

求集合乘积

集合中的元素都是: $x = g^e$

所以集合内所有元素的乘积为:

$$\prod x = \prod g^e = g^{\sum e}$$

因此只需要考虑所有满足条件的指数 e 的和。

1006 真的是原根?

当 $d > 2$ 时, 满足 $\gcd(t, d) = 1$ 的 t 会成对出现: t 和 $d - t$

并且有:

$$\gcd(t, d) = 1 \Rightarrow \gcd(d - t, d) = 1$$

每一对的和都是 d , 所以所有互质的 t 的平均值为: $\frac{d}{2}$

个数为: $\varphi(d)$

1006 真的是原根?

因此: $\sum t = \frac{d\varphi(d)}{2}$

又因为: $e = kt$

所以: $\sum e = k \sum t = k \cdot \frac{d\varphi(d)}{2}$

由于: $kd = n = p - 1$

所以: $\sum e = \frac{(p-1)\varphi(d)}{2}$

1007 色拉的美刀让饱饱不开心

先想一个问题：这些新来的 p 美刀有没有必要分开放？其实没有。
这个猜也能猜到。

把每张卡多加的钱记成 d_i ，那么任意一个区间的贡献都是

$$\max(a_l + d_l, a_{l+1} + d_{l+1}, \dots, a_r + d_r),$$

这是若干个一次函数取最大值，所以是凸函数；所有区间加起来仍然是凸函数。而我们限制的是 $\sum d_i = p, d_i \geq 0$ ，这是一个单纯形。凸函数在这种区域上取最大值时，总能在某个顶点取到不差的值，也就是把所有钱都放到同一张卡上。

1007 色拉的美刀让饱饱不开心

直观一点说，如果一个方案把钱分给了两张卡，那么只看这两张卡，把钱在它们之间左右挪动，答案作为这一维上的函数仍然是凸的，所以最大值不会出现在中间，至少有一端不更差。一直这样推下去，就能把钱全推到某一个位置。

于是每次询问只需要枚举把所有 p 加到哪张卡。先把原数组所有区间最大值之和记为 ans 。如果把钱都加到位置 x ，那么只有包含 x 的区间可能改变。一个包含 x 的区间原来的最大值如果已经大于 $a_x + p$ ，那它不会变；否则它的新最大值就会被 x 更新成 $a_x + p$ 。

1007 色拉的美刀让饱饱不开心

所以对固定的 x ，我们只要知道在阈值 $a_x + p$ 下， x 左右最多能扩出去多远，使得这一整段里的数都不超过这个阈值。设左边能扩 L 个，右边能扩 R 个，那么会被 x 接管的区间一共有 $(L + 1)(R + 1)$ 个，它们的新贡献总和是

$$(L + 1)(R + 1)(a_x + p).$$

但是这些区间在原数组里本来也有贡献，所以还要减掉它们原来的区间最大值之和。

1007 色拉的美刀让饱饱不开心

接下来预处理这个原贡献即可。先求出任意区间的最大值，并顺便对一端做前缀和，这样之后求一批“固定一端、另一端在一段范围内变化”的区间最大值总和可以直接得到。然后对每个中心 x ，按值从小到大模拟合法段的扩张。当前左右两边各有一个待加入的位置，显然下一次会先加入值更小的一边，因为它对应的阈值更早被满足。每加入一个新位置，就会多出一批包含 x 且经过这个新位置的区间，把这些区间原来的最大值总和加进去，就能得到 $S_x(k)$ ：以 x 为中心，合法段大小扩到 k 时，所有相关区间的原最大值之和。

1007 色拉的美刀让饱饱不开心

询问时枚举位置 x 。用预处理出的左右前缀最大值二分，找到 $a_x + p$ 能覆盖的左扩长度 L 和右扩长度 R 。那么把钱全加到 x 的答案就是

$$ans + (L + 1)(R + 1)(a_x + p) - S_x(L + R).$$

对所有 x 取最大值，就是这次询问的答案。

预处理是 $O(n^2)$ ，每次询问枚举 x 并二分左右范围，复杂度 $O(n \log n)$ 。由于 $n, q \leq 1000$ ，这个复杂度是够的。

1008 星际贸易市场

思路

本题需要支持两种操作：

1. 区间加法；
2. 查询区间内所有两两乘积之和。

也就是对于区间 $[l, r]$ ，要求：

$$\sum (a[i] \times a[j]) (l \leq i < j \leq r)$$

由于既有区间修改，又有区间查询，所以可以使用线段树维护。

对于线段树的每个节点，维护两个值：

- sum ：当前区间所有数的和；
- ans ：当前区间所有两两乘积之和。

1008 星际贸易市场

状态维护

假设当前区间被分成左右两个子区间。

左区间为 L ，右区间为 R 。

那么当前区间的和为：

$$sum = L.sum + R.sum;$$

当前区间的两两乘积和为：

$$ans = L.ans + R.ans + L.sum * R.sum;$$

1008 星际贸易市场

区间加法

如果一个长度为 len 的区间，每个数都加上 d 。

原来的区间和为 sum ，两两乘积和为 ans 。

对于任意一对数：

$$(a_i + d) \times (a_j + d) = a_i \times a_j + d \times a_i + d \times a_j + d \times d$$

对于整个区间，更新后的两两乘积和为：

$$ans += d \times (len - 1) \times sum + C(len, 2) \times d \times d$$

同时区间和变为： $sum += len \times d$

1008 星际贸易市场

复杂度分析

设数组长度为 n ，操作次数为 q 。

每次修改和查询的复杂度都是： $O(\log n)$

所以总时间复杂度为： $O(q \log n)$

空间复杂度为： $O(n)$

1009 大户爱的蚂蚁题

令顺时针为正方向，逆时针为负方向。

- 引理 1：运动结束后起点位置和终点位置集合相同。

证明：考虑到经典的转化，两只蚂蚁相遇之后会穿过去（每只蚂蚁视作相同）。所以每只蚂蚁都没有阻挡了，最后一定会回到原点。

- 推论 1：

设 m 只蚂蚁走正方向，则 $n - m$ 只蚂蚁走反方向。总位移为 $2m - n$ 个单位。

1009 大户爱的蚂蚁题

- 引理 2: 每只蚂蚁的相对位置不变。

证明: 考虑两只蚂蚁碰到之后会弹回去, 所以相对位置一定不会发生变化。

- 推论 2:

假设每只蚂蚁的起点为 s_i , 终点为 s_{end_i} (由于引理 1, 这是肯定可以表达出来的)。则 $end_i - i$ 全部相同。因为相对位置不变。

1009 大户爱的蚂蚁题

枚举 $k = end_i - i$, 则总位移为 k 。考虑到每只蚂蚁从起点移到下一只蚂蚁的起点, 和恰好为 1 单位长度。所以每只蚂蚁移到后第 k 只蚂蚁的起点, 位移总和为 k 。

所以有了一个等式:

$$2m - n = k$$

要求回到原点, k 一定为 0 或 $-n$ 或 n , 所以 $2m - n \equiv 0 \pmod n$ 。

如果 n 为奇数, $m = 0$ 或 $m = n$ 。概率为 $\frac{1}{2^{n-1}}$ 。

如果 n 为偶数, m 还可以为 $\frac{n}{2}$, 令 $x = \frac{n}{2}$, 概率为 $\frac{1}{2^{n-1}} + \frac{\binom{n}{x}}{2^n}$ 。

1010 留下(LiuXia)

判断相邻两项是否合法，只和它们的符号有关。令

$$g_i(x) = [(a_i \oplus x) < b_i],$$

也就是 $f(i)$ 是否为负。那么如果两个相邻位置一个为负一个为正，就一定合法；如果其中某一个刚好等于 0，也一定合法。

所以对相邻的 $i, i+1$ ，它合法的 x 可以分成两部分：一部分是 $g_i(x)$ 和 $g_{i+1}(x)$ 不同，另一部分是 $f(i) = 0$ 或 $f(i+1) = 0$ 。

1010 留下(LiuXia)

于是问题变成了：对每条相邻边 i ，求出哪些询问 x 会让它合法。我们按 $i = 1, 2, \dots, n - 1$ 从左到右处理，如果某个询问第一次被一条边覆盖，那么这条边就是它最左侧的答案，之后就不用再管它了。

现在看固定的一个位置 i ，集合

$$(a_i \oplus x) < b_i$$

在 x 上长什么样。可以先令 $y = a_i \oplus x$ ，那么条件变成 $y < b_i$ 。所有小于 b_i 的 y ，可以按照二进制前缀拆成 $O(\log V)$ 个连续块：由于 $x = a_i \oplus y$ ，当 y 的高位前缀固定、低位任意时，异或回去以后仍然会变成一段连续的 x 区间。

1010 留下(LiuXia)

所以每个位置的“负号集合”都能拆成 $O(\log V)$ 个区间，这里 $V = 2^{30}$ 。

对于相邻的两个位置 $i, i + 1$ ，严格异号正好就是这两个负号集合的对称差。把两个区间集合的端点从小到大扫一遍，就可以求出它们的对称差区间。除此之外，还要把等于零的情况单独加入： $f(i) = 0$ 当且仅当

$$x = a_i \oplus b_i,$$

1010 留下(LiuXia)

同理 $f(i+1) = 0$ 对应 $x = a_{i+1} \oplus b_{i+1}$ 。这些点也都能让这条边合法。

剩下就是离线处理询问：先把所有询问的 x 排序去重，只在这些点上赋答案。每处理一条边，就拿它对应的若干合法区间去覆盖这些询问点；如果某个点已经有答案，就跳过。可以用并查集维护“下一个还没有答案的位置”，每次覆盖区间时从第一个不小于左端点的询问开始，赋完一个就把它并到下一个位置。这样每个询问点最多被赋值一次。

1010 留下(LiuXia)

因为边是从左到右处理的，所以一个询问第一次被覆盖时得到的编号一定就是最左的合法 i 。如果所有边处理完它仍然没有被覆盖，答案就是 -1 。

每个位置的负号集合有 $O(\log V)$ 个区间，每条边求一次对称差也是 $O(\log V)$ 级别。每个询问只会被赋值一次，总复杂度可以看成

$$O((n \log V + q)\alpha(q) + q \log q),$$

其中 $V = 2^{30}$ ，可以通过本题的数据范围。